

LAB 1.2: VPC, SUBNETS & NETWORK SETUP FOR DATA PLATFORM

PART 0: UNDERSTANDING WHY NETWORKING MATTERS

Before we start building, let's understand why this lab is critical for your career.

The Big Picture Story

Imagine you're building a data engineering platform for a major bank:

The Bank's Requirements:

- Customers' financial data must NEVER be exposed to the internet
- Database servers must be locked down (no one can directly access)
- Only authorized applications can read/write data
- Auditors need to track who accessed what
- Data must travel securely (encrypted)

What happens without proper networking?

Bad Scenario (No VPC Security):

1. Someone accidentally opens database port to entire internet
2. Hackers scan for open ports (automated, happens in minutes)
3. They find your database exposed
4. They download all customer data
5. You're on the news, company goes bankrupt, you're unemployed

Good Scenario (With VPC):

1. Database is in PRIVATE subnet (not accessible from internet)
2. Even if someone tries, they can't reach it
3. Hackers find nothing
4. Your data is safe
5. Company thrives, you get promoted

Real companies have learned this the hard way:

- 2013: Target breach - 40 million credit cards stolen - \$18.5 million settlement
- 2016: Yahoo breach - 500 million accounts compromised
- 2017: Equifax breach - 147 million people's data stolen

Why? Poor network security.

What Is A VPC? (In Plain English)

VPC = Virtual Private Cloud

Think of it like this:

Your AWS Account (A real estate development)

```

|
└─ VPC (Your private land - 10.0.0.0/16 = 65,536 addresses)
    |
    └─ Public Subnet (10.0.1.0/24)
        |
        └─ Like: Downtown area (accessible from street)
        |
        └─ Contains: NAT Gateway (the only door to internet)
        |
        └─ Security: Firewall at the door
        |
    └─ Private Subnet 1 (10.0.2.0/24)
        |
        └─ Like: Locked office building (no street access)
        |
        └─ Contains: Databases, sensitive data
        |
        └─ Access: Only through NAT Gateway
        |
    └─ Private Subnet 2 (10.0.3.0/24)
        |
        └─ Like: Secure server room
        |
        └─ Contains: Application servers, processing
        |
        └─ Access: Only through NAT Gateway
        |
    └─ Internet Gateway (The main gate)
        |
        └─ Controls: Traffic to/from internet
        |
    └─ NAT Gateway (The secure door)
        |
        └─ Purpose: Private servers CAN reach internet (downloads, updates)
        |
        └─ But: Internet CANNOT reach private servers
        |
        └─ Cost: $0.32/hour (why we delete it!)
        |
    └─ VPC Endpoints (Secret passages)
        |
        └─ S3 Endpoint: Private servers can reach S3 without internet
        |
        └─ DynamoDB Endpoint: Private servers can reach DynamoDB without internet
        |
        └─ Cost: FREE (why we use them!)
        |
    └─ Security Groups (Guards at each door)
        |
        └─ Port 443 (HTTPS): Allowed
        |
        └─ Port 22 (SSH): Only from specific IPs
        |
        └─ Database ports: Only from application servers
        |
        └─ Everything else: DENIED
        |
    └─ Route Tables (GPS for traffic)
        |
        └─ Public: "Go to internet if you don't recognize the address"
        |
        └─ Private: "Go to NAT Gateway for internet, stay local otherwise"

```

Real-World Example: Netflix Architecture

Netflix handles 200 million users. Here's why they use VPCs:

Netflix VPC Architecture:

- └ Public Subnets
 - | └ Load Balancers (accept incoming requests from users)
 - |
- └ Private Subnets (Application Tier)
 - | └ App servers run here (can go OUT to internet, not IN)
 - |
- └ Private Subnets (Database Tier)
 - | └ Databases here (no internet access at all)
 - | └ Only reachable from application tier
 - |
- └ VPC Endpoints
 - | └ Apps reach AWS services without exposing to internet

If hackers breach the load balancers, they CANNOT reach the databases (they're in a different subnet).

Why This Matters For Your Career

1. Every company uses VPCs

- Day 1 at Amazon/Google/Microsoft: You'll use their VPCs
- Job interviews ask: "Design a secure network"
- This lab teaches you how

2. Network security = job security

- Companies pay premium for security engineers
- If you understand networks, you're valuable
- Skills that prevent breaches = career advancement

3. This prevents disasters

- Breaches cost millions
- Your company avoids breaches = stability
- You prevent disasters = you get promoted

4. You'll explain this to non-technical people

- CEO asks: "How is our data protected?"
- You answer: "It's in a private subnet, only accessible through approved channels"
- CEO is satisfied, company is safe

PART 1: GOALS FOR THIS LAB

By the end of this lab, you will:

- **Understand** how network security works in AWS
- **Explain** why we separate public and private subnets
- **Design** network architecture for a data platform
- **Create** a production-ready VPC with security
- **Optimize** for cost (using VPC endpoints instead of NAT)

- **Know** when to use this architecture in real jobs

PART 2: WHAT YOU'LL CREATE

The Complete Network We're Building

```
VPC: data-platform-vpc (10.0.0.0/16)
|
├─ PUBLIC SUBNET (10.0.1.0/24) - "The Lobby"
|   └─ NAT Gateway here (costs $0.32/hour when running)
|   └─ Internet Gateway (free)
|   └─ Security Group: sg-public-nat
|
├─ PRIVATE SUBNET 1 (10.0.2.0/24, us-east-1a) - "Database Building"
|   └─ RDS databases live here (no internet access)
|   └─ Security Group: sg-private-db
|
├─ PRIVATE SUBNET 2 (10.0.3.0/24, us-east-1b) - "Compute Building"
|   └─ EC2, Lambda, Glue run here
|   └─ Security Group: sg-private-compute
|
├─ ROUTE TABLE: Public Route
|   └─ Unknown traffic → Internet Gateway
|
├─ ROUTE TABLE: Private Route
|   └─ Unknown traffic → NAT Gateway
|   └─ Associated with BOTH private subnets
|
├─ SECURITY GROUPS (Firewalls)
|   ├── sg-public-nat: Allow HTTPS from anywhere
|   ├── sg-private-compute: Allow all traffic from itself + public SG
|   └─ sg-private-db: Allow MySQL/PostgreSQL only from compute SG
|
└─ VPC ENDPOINTS (Secret passages to avoid NAT costs)
    ├── S3 Gateway Endpoint (FREE - save on data transfer!)
    ├── DynamoDB Gateway Endpoint (FREE - save on data transfer!)
    └─ Secrets Manager Interface Endpoint (CHEAP - encrypted access)
```

Why This Design?

Databases in Private Subnet:

- ☒ Not reachable from internet (secure)
- ☒ Only reachable from application servers (isolated)
- ☒ Can't download updates from internet (solution: NAT)

Applications in Private Subnet:

- ☒ Can reach internet (for updates, downloads)
- ☒ Can't be directly accessed from internet (secure)
- ☒ Can reach databases (in same VPC)

VPC Endpoints Instead of NAT:

- ☒ Reach AWS services without NAT Gateway
- ☒ Save money (FREE vs \$0.32/hour)
- ☒ More secure (no internet traffic)
- ☒ Faster (shorter path)

PART 3: THE "WHY" BEHIND DESIGN DECISIONS

Why Separate Public and Private Subnets?

Bad Design (Everything Public):

Internet ↔ [Databases] ↔ [Apps] ↔ [Load Balancers]

Problem: Hackers can reach databases directly!

Result: Data breach in minutes

Good Design (Layered Security):

Internet ↔ [Public LB] → [Private Apps] → [Private DBs]

Problem for hacker: Can only reach LB, not apps or DBs

Result: Breaches contained to load balancer

Why Two Private Subnets in Different AZs?

Availability Zones (AZs):

- Different data centers in same region
- If one catches fire, the other is fine
- AWS best practice: Always use 2+ AZs for redundancy

us-east-1a (Availability Zone A)

└─ Public Subnet

└─ Private Subnet 1 (Databases primary)

us-east-1b (Availability Zone B)

└─ Private Subnet 2 (Databases replica)

If AZ-A fails, AZ-B keeps your data safe.

Why VPC Endpoints?

Scenario without Endpoints:

Private App Server needs to access S3

└─ Data: Private Subnet → NAT Gateway → Internet Gateway → S3

└─ Path: Through internet (technically through AWS network, but slow)

└─ Cost: \$0.32/hour for NAT, PLUS \$0.02/GB data transfer

└─ Speed: slower (longer path)

└─ Security: Data travels through more hops

Scenario with VPC Endpoints:

```
Private App Server needs to access S3
├ Data: Private Subnet → S3 VPC Endpoint → S3
├ Path: Direct to AWS service (no internet)
├ Cost: FREE (or $0.01/hour for interface endpoints)
├ Speed: Faster (direct path)
└ Security: Data never leaves AWS network
```

Real numbers for your company:

- 100 app servers
- Each downloads 1GB/day from S3
- $100\text{GB/day} \times \$0.02/\text{GB} = \$2/\text{day} = \$60/\text{month}$

Without VPC Endpoint: \$60/month data transfer + \$230/month NAT = \$290/month

With VPC Endpoint: \$30/month (endpoint cost) = \$260/month SAVED

Why Security Groups Are Crucial

Scenario without Security Groups:

```
All ports open on all servers
├ Hacker finds database port (3306)
├ Hacker tries to connect
├ No firewall stops them
└ They're in! Data stolen.
```

Scenario with Security Groups:

```
Database server SG only allows port 3306 FROM app server SG
├ Hacker tries to connect from internet → BLOCKED
├ Hacker compromises app server → CAN connect to DB
├ But: Damage is contained to app server
└ Monitoring alerts you, you shut down app server
```

Key principle: "Deny by default, allow explicitly"

PART 4: PREREQUISITE - CHECK THESE FIRST

Before starting, make sure you have:

1. **AWS Account** - Must be same account as Lab 1.1
2. **AWS Management Console Access** - You can log in at <https://console.aws.amazon.com>
3. **Admin or PowerUser access** - You need VPC creation permissions
4. **IAM Roles from Lab 1.1** - You should have created these already
5. **Text Editor** - Notepad, VS Code (to save VPC IDs, subnet IDs, etc.)
6. **Time** - 4 uninterrupted hours
7. **Completion of Lab 1.1** - IAM setup must be done first (required for roles)

Don't have these? Go back to Lab 1.1 first.

STEP 0: LOGIN AND PREPARE

Step 0.1: Login to AWS

1. **Open:** <https://console.aws.amazon.com>
2. **Enter credentials** (same as Lab 1.1)
3. **If prompted, enter MFA code** from your authenticator app
4. **Verify logged in** - See account name in top right

Step 0.2: Verify Region

1. **Top right corner, check region dropdown** (might say "N. Virginia", "us-east-1", etc.)
2. **Must be:** `us-east-1` (US East N. Virginia)
 - Why? Cost consistency, service availability, same as Lab 1.1
3. **If different, click and select:** `us-east-1`
4. **Confirm region changed**

Expected Result: Region shows "us-east-1" in top right

Step 0.3: Prepare Documentation File

1. **Open text editor** (Notepad, VS Code, etc.)
2. **Create new file:** `Lab_1_2_VPC_Setup.txt`
3. **Save it** somewhere you'll remember
4. **You'll copy VPC IDs, subnet IDs, etc. into this file as you go**

Why? You'll need these IDs in Lab 1.3 and later labs. Saves time vs. searching AWS console.

PART 1: CREATE VPC

Why This Order?

We create resources in this order because each depends on the previous:

1. VPC first (it's the container for everything)
 2. Subnets inside VPC (need VPC to exist first)
 3. Gateways (Internet Gateway)
 4. NAT/Endpoints (depend on subnets)
 5. Route tables (depend on gateways)
 6. Security groups (last, can work alone)
-

Step 1.1: Navigate to VPC Console

1. Click **"Services"** (top left, menu icon)
2. Type in search: `vpc`
3. Click **"VPC"** from results (says "Virtual Private Cloud")
4. You're now in **VPC Console**

What you should see:

- Left sidebar with options: "Your VPCs", "Subnets", "Internet Gateways", etc.
- Main area showing "VPC Dashboard"

Why the VPC Console?

- This is where ALL network infrastructure is managed
- Central location for security configuration
- Shows everything together (unlike EC2 console which is messy)

Step 1.2: Create a New VPC

1. Click **"Your VPCs"** in left sidebar
2. Click blue **"Create VPC"** button (top right)
3. You're on **"Create VPC"** page

Step 1.3: Enter VPC Details

VPC Settings Section:

Field 1: Name tag

- **Type:** `data-platform-vpc`
- **Why:** Clear, professional name that describes purpose

Field 2: IPv4 CIDR block

- **Type:** `10.0.0.0/16`
- **What does this mean?**
 - `10.0.0.0` = Starting address
 - `/16` = Network mask (means: from 10.0.0.0 to 10.0.255.255)
 - **Total:** 65,536 possible IP addresses
- **Why 10.0.0.0/16?**
 - RFC 1918 private range (not internet routable)
 - Standard for internal networks
 - Large enough for most companies
 - Small enough to manage easily

Field 3: IPv6 CIDR block

- **Leave as:** "No IPv6 CIDR Block"
- **Why?** IPv4 is standard for data engineering (IPv6 is advanced)

Field 4: Tenancy

- **Keep as:** "Default"
 - **Why?** Default is for almost everyone (Dedicated is for special cases)
4. Click **"Create VPC"** button (blue, bottom right)

Expected Result:

- Success message: "Successfully created VPC"
- Redirected to VPC details page
- Shows: Name, ID (vpc-xxxxx), CIDR block (10.0.0.0/16)

Copy and save:

```
VPC ID: vpc-[COPY THIS]
CIDR: 10.0.0.0/16
```

PART 2: CREATE SUBNETS

Remember: Subnets are like neighborhoods within the VPC

Why 3 Subnets?

```
Public Subnet (1):
└─ NAT Gateway (costs money)
└─ Internet Gateway
└─ Only 1 needed (it's stateless)

Private Subnets (2):
└─ Databases (1a)
└─ Applications (1b)
└─ 2 recommended (different AZs for redundancy)
```

Subnet 1: Public Subnet (for NAT Gateway)

Step 2.1: Create Public Subnet

1. Click **"Subnets"** in left sidebar
2. Click blue **"Create subnet"** button
3. You're on **"Create subnet"** page

Step 2.2: Fill in Subnet Details

Field 1: VPC ID

- Click **dropdown**, select `data-platform-vpc`
- **Why?** This subnet belongs to our VPC

Field 2: Subnet name

- **Type:** `public-subnet-1a`

- **Why:** "public" = will route to internet, "1a" = availability zone

Field 3: Availability Zone

- **Click dropdown**, select `us-east-1a`
- **Why?** First AZ in region us-east-1

Field 4: IPv4 subnet CIDR block

- **Type:** `10.0.1.0/24`
- **What does this mean?**
 - `/24` = 256 IP addresses (10.0.1.0 to 10.0.1.255)
 - This is "slicing" our VPC (10.0.0.0/16) into smaller pieces
 - `/24` is standard for subnets
- **Why 10.0.1.0/24?**
 - Fits within our VPC range (10.0.0.0/16)
 - 256 IPs is typical for a subnet
 - Easy to remember (1st octet = 1)

5. **Click "Create subnet"** (blue button)

Expected Result: Success message, subnet appears in list

Copy and save:

```
Public Subnet ID: subnet-[COPY THIS]
Public Subnet CIDR: 10.0.1.0/24
```

Subnet 2: Private Subnet 1 (for Databases)

Step 2.3: Create First Private Subnet

1. **Click "Create subnet"** (blue button again)

Field 1: VPC ID

- **Select:** `data-platform-vpc`

Field 2: Subnet name

- **Type:** `private-subnet-1a`

Field 3: Availability Zone

- **Select:** `us-east-1a` (same AZ as public for convenience)

Field 4: IPv4 subnet CIDR block

- **Type:** `10.0.2.0/24`
- **Why .2.0 and not .1.0?**
 - .1.0 is our public subnet
 - .2.0 is our first private

- .3.0 will be our second private
- Avoids overlap

2. Click "Create subnet"

Expected Result: Success message

Copy and save:

```
Private Subnet 1A ID: subnet-[COPY THIS]
Private Subnet 1A CIDR: 10.0.2.0/24
```

Subnet 3: Private Subnet 2 (for Applications)

Step 2.4: Create Second Private Subnet

1. Click "Create subnet" (blue button again)

Field 1: VPC ID

- **Select:** `data-platform-vpc`

Field 2: Subnet name

- **Type:** `private-subnet-1b`
- **Why "1b"?** Different availability zone for redundancy

Field 3: Availability Zone

- **Select:** `us-east-1b` (DIFFERENT from previous!)
- **Why different AZ?**
 - AWS best practice: Always use 2+ AZs
 - If 1a fails, 1b keeps running
 - True redundancy

Field 4: IPv4 subnet CIDR block

- **Type:** `10.0.3.0/24`
- **Why .3.0?**
 - .1.0 = public subnet
 - .2.0 = first private
 - .3.0 = second private

2. Click "Create subnet"

Expected Result: Success message

Copy and save:

```
Private Subnet 1B ID: subnet-[COPY THIS]
Private Subnet 1B CIDR: 10.0.3.0/24
```

Step 2.5: Verify All Subnets Created

1. Stay on "Subnets" page
2. You should see 3 subnets in the list:

Name	CIDR	AZ	Status
public-subnet-1a	10.0.1.0/24	us-east-1a	Available
private-subnet-1a	10.0.2.0/24	us-east-1a	Available
private-subnet-1b	10.0.3.0/24	us-east-1b	Available

If all 3 are there with "Available" status: ✓ Good!

What you just did:

- Created a VPC (virtual private cloud)
- Divided it into 3 subnets
- Positioned them in 2 different availability zones
- Set up redundancy (if one AZ fails, you have another)

PART 3: CREATE INTERNET GATEWAY

What's an Internet Gateway?

Think of it like a border checkpoint:

- Incoming traffic from internet → Checked by IGW
- Outgoing traffic to internet → Checked by IGW
- Not a firewall (security groups are), just a gateway

Step 3.1: Navigate to Internet Gateways

1. Click "Internet gateways" in left sidebar
2. Click blue "Create internet gateway" button

Step 3.2: Create Internet Gateway

1. Name tag: `data-platform-igw`
2. Click "Create internet gateway" (blue button)

Expected Result: Success message, IGW created

Step 3.3: Attach IGW to VPC

Why attach?

- IGW created but not connected to your VPC yet
- Like installing a door but not connecting it to the wall
- Must explicitly attach

After creation, you should see a message "Attach to a VPC" with a blue button.

1. Click **"Attach to VPC"** (blue button)
2. **VPC dropdown:** Select `data-platform-vpc`
3. Click **"Attach internet gateway"** (blue button)

Expected Result: Success message, IGW now shows "State: Attached"

Copy and save:

```
Internet Gateway ID: igw-[COPY THIS]
```

PART 4: ALLOCATE ELASTIC IP FOR NAT GATEWAY

What's an Elastic IP?

NAT Gateway needs a static public IP address. Like your home has a fixed address, not a random one.

Step 4.1: Navigate to Elastic IPs

1. Click **"Elastic IPs"** in left sidebar
2. Click blue **"Allocate Elastic IP address"** button

Step 4.2: Allocate IP

Form fields:

Field 1: Network Border Group

- **Keep as:** (default, should be your region)

Field 2: Public IPv4 address pool

- **Keep as:** "Amazon's pool of IP addresses"
3. Click **"Allocate"** (blue button)

Expected Result:

- Success message
- You see an Elastic IP assigned (like 54.123.45.67)

Copy and save:

```
Elastic IP: [IP ADDRESS - copy this!]  
Allocation ID: eipalloc-[COPY THIS]
```

Why save this?

- You'll need it for NAT Gateway setup next
 - Also good for documentation
-

PART 5: CREATE NAT GATEWAY

What's NAT? (Network Address Translation)

NAT Gateway allows private servers to reach the internet for updates/downloads, but prevents the internet from reaching them.

Without NAT:

- └ Private servers: Can't reach internet (can't download updates!)
- └ Internet: Can't reach servers (safe but isolated)

With NAT:

- └ Private servers: CAN reach internet through NAT (updates work!)
- └ Internet: CANNOT reach servers directly (still safe!)

Step 5.1: Navigate to NAT Gateways

1. Click "NAT gateways" in left sidebar
2. Click blue "Create NAT gateway" button
3. You're on "Create NAT gateway" page

Step 5.2: Configure NAT Gateway

Field 1: Name

- **Type:** `data-platform-nat`

Field 2: Subnet

- Click **dropdown**, select `public-subnet-1a`
- **IMPORTANT:** NAT Gateway MUST be in PUBLIC subnet!
- **Why?** It needs direct access to Internet Gateway
- If you put it in private subnet, it can't reach internet!

Field 3: Elastic IP allocation ID

- Click **dropdown**, select the Elastic IP you just created
- **It should show:** `eipalloc-xxx | 54.xxx.xxx.xxx`
- **This connects the Elastic IP to the NAT Gateway**

4. Click "Create NAT gateway" (blue button)

Expected Result:

- Success message
- Status shows "Pending" then "Available" (takes 1-2 minutes)

Copy and save:

NAT Gateway ID: nat-[COPY THIS]
Elastic IP: [IP ADDRESS]

PART 6: CREATE ROUTE TABLES

What's a Route Table?

Route table = GPS for your traffic

Routes tell traffic where to go:

- └ "Traffic to 10.0.0.0/16? Stay local (same VPC)"
- └ "Traffic to anywhere else? Go to Internet Gateway"
- └ "Or: Go to NAT Gateway" (for private subnets)

Route Table 1: Public Route Table

Purpose: Route table for public subnet

Step 6.1: Create Public Route Table

1. Click **"Route tables"** in left sidebar
2. Click blue **"Create route table"** button

Step 6.2: Configure Public Route Table

1. **Name:** `public-route-table`
2. **VPC:** Select `data-platform-vpc` from dropdown
3. Click **"Create route table"** (blue button)

Expected Result: Route table created, you're on its details page

Step 6.3: Add Route to Internet Gateway

1. Scroll to **"Routes"** section
2. Click **"Edit routes"** button

Current routes should show:

Destination	Target
10.0.0.0/16	Local (traffic within VPC stays local)

3. Click **"Add route"** button
4. **Fill in new route:**
 - **Destination:** `0.0.0.0/0` (means "all traffic not destined for VPC")
 - **Target:** Click dropdown, select "Internet Gateway"
 - **Target (continued):** Then select `data-platform-igw`
5. Click **"Save routes"** (blue button)

Expected result: Route is added, shows:

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	data-platform-igw

What this means:

- Any traffic not going to 10.0.0.0/16 (our VPC) → Internet Gateway
- Internet Gateway handles connection to internet

Step 6.4: Associate with Public Subnet

1. Scroll down to "Subnet associations"
2. Click "Edit subnet associations" button
3. Check the box for: `public-subnet-1a`
4. Click "Save associations" (blue button)

Expected Result: "public-subnet-1a" now shows "Explicitly associated"

What you did:

- Created route table
- Added rule: "Send unknown traffic to Internet Gateway"
- Associated it with public subnet
- Now public subnet knows how to reach internet!

Copy and save:

Public Route Table ID: `rtb-[COPY THIS]`

Route Table 2: Private Route Table

Purpose: Route table for private subnets (sends unknown traffic to NAT, not Internet)

Step 6.5: Create Private Route Table

1. Click "Route tables" in left sidebar
2. Click blue "Create route table" button

Step 6.6: Configure Private Route Table

1. **Name:** `private-route-table`
2. **VPC:** Select `data-platform-vpc`
3. Click "Create route table"

Expected Result: Private route table created

Step 6.7: Add Route to NAT Gateway

1. Scroll to "Routes" section
2. Click "Edit routes" button
3. Click "Add route" button
4. Fill in new route:
 - **Destination:** `0.0.0.0/0` (all unknown traffic)
 - **Target:** Click dropdown, select "NAT Gateway"

- **Target (continued):** Then select `data-platform-nat`

5. Click "Save routes"

Expected Result: Route is added, shows:

Destination	Target
10.0.0.0/16	Local
0.0.0.0/0	data-platform-nat

What this means:

- Private subnets send unknown traffic to NAT Gateway
- NAT Gateway forwards to Internet Gateway
- Internet can't initiate connection back to private servers

Step 6.8: Associate with Private Subnets

1. Scroll down to "Subnet associations"

2. Click "Edit subnet associations"

3. Check boxes for BOTH:

- `private-subnet-1a`
- `private-subnet-1b`

4. Click "Save associations"

Expected Result: Both private subnets now show "Explicitly associated"

Copy and save:

Private Route Table ID: `rtb-[COPY THIS]`

PART 7: CREATE SECURITY GROUPS

What's a Security Group?

Security Group = Firewall for individual servers/groups

Without Security Groups:

- ├ All ports open on all servers
- └ Hackers can access anything

With Security Groups:

- ├ Only specified ports/sources allowed
 - ├ Everything else DENIED by default
 - └ Attacker can't find open ports
-

Security Group 1: Public SG (for NAT Gateway area)

Step 7.1: Navigate to Security Groups

1. Click "Security groups" in left sidebar
2. Click blue "Create security group" button

Step 7.2: Create Public Security Group

1. **Name:** sg-public-nat
2. **Description:** Security group for public subnet with NAT Gateway. Allows HTTPS inbound only.
3. **VPC:** Select data-platform-vpc

Step 7.3: Add Inbound Rules

Why HTTPS only?

- Port 443 = HTTPS (encrypted)
- Port 80 = HTTP (not encrypted, deprecated)
- We use only HTTPS for security

1. Scroll to "Inbound rules"
2. Click "Add rule" button
3. Fill in rule:
 - **Type:** HTTPS
 - **Source:** 0.0.0.0/0 (allow from anywhere)
 - **Description:** HTTPS from anywhere for secure connections
4. Click "Create security group" (blue button)

Expected Result: Security group created

Copy and save:

```
sg-public-nat ID: sg-[COPY THIS]
```

Security Group 2: Private Compute SG

Step 7.4: Create Private Compute Security Group

1. Click blue "Create security group" button
2. **Name:** sg-private-compute
3. **Description:** Security group for compute (EC2, Lambda, Glue) in private subnets. Allow internal traffic.
4. **VPC:** data-platform-vpc

Step 7.5: Add Inbound Rules for Private Compute

Rule 1: Allow from same security group

1. Scroll to "Inbound rules"
2. Click "Add rule"
3. Fill in:
 - **Type:** All traffic
 - **Source:** Click dropdown, select "Security Group"
 - **In the next field, type:** `sg-private-compute` (itself)
 - **Description:** `Allow all traffic from within this security group`
4. Click "Add rule" (green button)

Why this rule?

- Servers in compute subnet can talk to each other
- Example: App A can query App B
- Example: Load balancer can reach app servers

Rule 2: Allow from Public SG

5. Click "Add rule" again
6. Fill in:
 - **Type:** All traffic
 - **Source:** Security group dropdown
 - **Find and select:** `sg-public-nat`
 - **Description:** `Allow all traffic from public security group`
7. Click "Create security group"

Why this rule?

- NAT Gateway can send traffic to compute servers
- If you add load balancers later, they can send traffic here

Expected Result: Security group created

Copy and save:

```
sg-private-compute ID: sg-[COPY THIS]
```

Security Group 3: Private Database SG

Step 7.6: Create Private Database Security Group

1. Click blue "Create security group"
2. **Name:** `sg-private-db`
3. **Description:** `Security group for RDS databases in private subnets. Only allow from compute layer.`

4. **VPC:** `data-platform-vpc`

Step 7.7: Add Inbound Rules for Database

Rule 1: MySQL from Compute SG

1. Scroll to "Inbound rules"
2. Click "Add rule"
3. Fill in:
 - **Type:** MySQL/Aurora (3306)
 - **Source:** Security group dropdown
 - **Find and select:** `sg-private-compute`
 - **Description:** `MySQL from compute subnet`
4. Click "Add rule"

Why MySQL only?

- Port 3306 = MySQL database
- Only necessary port
- Hackers can't use PostgreSQL, HTTPS, SSH, etc.

Rule 2: PostgreSQL from Compute SG

5. Click "Add rule" again
6. Fill in:
 - **Type:** PostgreSQL (5432)
 - **Source:** Security group → `sg-private-compute`
 - **Description:** `PostgreSQL from compute subnet`
7. Click "Create security group"

Why both MySQL and PostgreSQL?

- Different projects use different databases
- Company might have both
- Better to allow both and use what's needed

Expected Result: Security group created

Copy and save:

```
sg-private-db ID: sg-[COPY THIS]
```

PART 8: CREATE VPC ENDPOINTS

Why VPC Endpoints?

Remember the cost calculation:

- NAT Gateway: \$230/month + \$60/month data transfer = \$290/month
- VPC Endpoints: \$30/month + FREE data transfer = \$30/month

- **Savings: \$260/month!**

VPC Endpoint 1: S3 Gateway Endpoint

Purpose: Private servers can access S3 without internet

Step 8.1: Navigate to VPC Endpoints

1. Click **"Endpoints"** in left sidebar
2. Click blue **"Create endpoint"** button
3. You're on **"Create endpoint"** page

Step 8.2: Configure S3 Endpoint

1. **Service category:** Keep "AWS services"
2. **Services search box:** Type `s3`
3. **Find and click:** `com.amazonaws.us-east-1.s3` (Gateway type)
 - **IMPORTANT:** Make sure it says "Gateway" not "Interface"
 - Gateway = Free or very cheap
 - Interface = Costs more
4. **VPC:** Select `data-platform-vpc` from dropdown

Step 8.3: Configure Route Tables

1. **Route table IDs section:** Shows checkboxes
2. **Check:** `private-route-table`
3. **Leave unchecked:** `public-route-table` (S3 endpoint not needed there)
4. **Policy:** Keep as "Full access"
5. Click **"Create endpoint"** (blue button)

Expected Result: Endpoint created, status shows "Available"

What this does:

- Private subnet route table gets new route: `s3.* → S3 VPC Endpoint`
- Now private servers can reach S3 without going through NAT!

Copy and save:

S3 Endpoint ID: `vpce-[COPY THIS]`

VPC Endpoint 2: DynamoDB Gateway Endpoint

Same process, different service

Step 8.4: Create DynamoDB Endpoint

1. Click blue "Create endpoint"
2. **Service category:** "AWS services"
3. **Services search:** Type `dynamodb`
4. **Find and click:** `com.amazonaws.us-east-1.dynamodb` (Gateway)
5. **VPC:** `data-platform-vpc`
6. **Route table IDs:** Check `private-route-table`
7. **Policy:** "Full access"
8. Click "Create endpoint"

Expected Result: Endpoint created, status "Available"

Copy and save:

```
DynamoDB Endpoint ID: vpce-[COPY THIS]
```

VPC Endpoint 3: Secrets Manager Interface Endpoint

Different type: Interface Endpoint

Why Interface instead of Gateway?

- S3 and DynamoDB have Gateway endpoints (free/cheap)
- Secrets Manager only has Interface endpoints
- Interface = Costs ~\$7/month (still cheaper than NAT!)

Step 8.5: Create Secrets Manager Endpoint

1. Click blue "Create endpoint"
2. **Service category:** "AWS services"
3. **Services search:** Type `secretsmanager`
4. **Find and click:** `com.amazonaws.us-east-1.secretsmanager` (Interface)
5. **VPC:** `data-platform-vpc`
6. **Subnets:** Check BOTH:
 - `private-subnet-1a`
 - `private-subnet-1b`
 - **Why both?** Redundancy across AZs
7. **Security group:** Select `sg-private-compute`
 - **Why?** Only compute servers need to access secrets
8. **Policy:** "Full access"
9. Click "Create endpoint"

Expected Result: Endpoint created, status "Available"

Copy and save:

PART 9: VERIFY YOUR COMPLETE NETWORK

Step 9.1: Verification Checklist

VPC:

- ☐ Click "Your VPCs"
- ☐ Find: `data-platform-vpc` with CIDR 10.0.0.0/16
- ☐ Status: Available

Subnets:

- ☐ Click "Subnets"
- ☐ See 3 subnets with correct CIDRs:
 - ☐ public-subnet-1a (10.0.1.0/24, us-east-1a)
 - ☐ private-subnet-1a (10.0.2.0/24, us-east-1a)
 - ☐ private-subnet-1b (10.0.3.0/24, us-east-1b)

Gateways:

- ☐ Click "Internet gateways"
- ☐ See: data-platform-igw, Status: Attached
- ☐ Click "NAT gateways"
- ☐ See: data-platform-nat, Status: Available

Route Tables:

- ☐ Click "Route tables"
- ☐ Public RT has: 10.0.0.0/16 → Local, 0.0.0.0/0 → IGW
- ☐ Private RT has: 10.0.0.0/16 → Local, 0.0.0.0/0 → NAT
- ☐ Associations correct for each

Security Groups:

- ☐ Click "Security groups"
- ☐ See: sg-public-nat, sg-private-compute, sg-private-db
- ☐ Each has correct rules

Endpoints:

- ☐ Click "Endpoints"
- ☐ See 3 endpoints, all status "Available"
- ☐ S3 Gateway, DynamoDB Gateway, Secrets Manager Interface

If all checks pass: ✓ Your network is correctly configured!

PART 10: DOCUMENT YOUR NETWORK

Create a file: `Lab_1_2_VPC_Documentation.txt`

```
=== LAB 1.2: VPC & NETWORK SETUP DOCUMENTATION ===  
Date Created: [TODAY'S DATE]
```

VPC DETAILS:

```
|─ VPC Name: data-platform-vpc  
|─ VPC ID: vpc-[PASTE HERE]  
|─ CIDR Block: 10.0.0.0/16 (65,536 IP addresses)  
└─ Region: us-east-1
```

SUBNETS:

```
|─ PUBLIC SUBNET (for NAT Gateway)  
| |─ Name: public-subnet-1a  
| |─ ID: subnet-[PASTE HERE]  
| |─ CIDR: 10.0.1.0/24  
| |─ AZ: us-east-1a  
| └─ Route Table: public-route-table (rtb-xxx)  
|  
|─ PRIVATE SUBNET 1 (for Databases)  
| |─ Name: private-subnet-1a  
| |─ ID: subnet-[PASTE HERE]  
| |─ CIDR: 10.0.2.0/24  
| |─ AZ: us-east-1a  
| └─ Route Table: private-route-table (rtb-xxx)  
|  
└─ PRIVATE SUBNET 2 (for Applications)  
  |─ Name: private-subnet-1b  
  |─ ID: subnet-[PASTE HERE]  
  |─ CIDR: 10.0.3.0/24  
  |─ AZ: us-east-1b  
  └─ Route Table: private-route-table (rtb-xxx)
```

INTERNET CONNECTIVITY:

```
|─ Internet Gateway  
| |─ Name: data-platform-igw  
| |─ ID: igw-[PASTE HERE]  
| └─ Status: Attached  
|  
└─ NAT Gateway  
  |─ Name: data-platform-nat  
  |─ ID: nat-[PASTE HERE]  
  |─ Elastic IP: [PASTE IP ADDRESS]  
  |─ Status: Available  
  └─ Cost: $0.32/hour (DELETE when not in use!)
```

SECURITY GROUPS:

```
|─ sg-public-nat  
| |─ ID: sg-[PASTE HERE]  
| └─ Rules: HTTPS inbound from anywhere  
|  
|─ sg-private-compute  
| |─ ID: sg-[PASTE HERE]
```


- | └ Rules: All traffic from itself + public-nat
- |
- └ sg-private-db
 - | └ ID: sg-[PASTE HERE]
 - └ Rules: MySQL 3306 + PostgreSQL 5432 from compute

VPC ENDPOINTS:

- | └ S3 Gateway
 - | └ ID: vpce-[PASTE HERE]
 - | └ Cost: FREE
 - | └ Associated with: private-route-table
- |
- | └ DynamoDB Gateway
 - | └ ID: vpce-[PASTE HERE]
 - | └ Cost: FREE
 - | └ Associated with: private-route-table
- |
- └ Secrets Manager Interface
 - | └ ID: vpce-[PASTE HERE]
 - | └ Cost: ~\$7/month
 - └ Subnets: private-subnet-1a, private-subnet-1b

ROUTE TABLES:

- | └ Public Route Table (rtb-xxx)
 - | └ ID: rtb-[PASTE HERE]
 - | └ Routes:
 - | | └ 10.0.0.0/16 → Local
 - | | └ 0.0.0.0/0 → Internet Gateway (igw-xxx)
 - | └ Associated Subnets: public-subnet-1a
- |
- └ Private Route Table (rtb-xxx)
 - | └ ID: rtb-[PASTE HERE]
 - | └ Routes:
 - | | └ 10.0.0.0/16 → Local
 - | | └ 0.0.0.0/0 → NAT Gateway (nat-xxx)
 - └ Associated Subnets: private-subnet-1a, private-subnet-1b

SECURITY ARCHITECTURE:

- | └ Internet → IGW → Public Subnet → NAT → Private Subnets
- | └ Private servers can reach internet for updates (via NAT)
- | └ Internet CANNOT reach private servers directly
- └ Databases only accessible from application servers

NETWORK TRAFFIC FLOW:

- | └ Inbound: Internet → IGW → Public SG → (no further allowed)
- | └ Outbound (from Private Apps): Apps → NAT → IGW → Internet
- | └ within VPC: All subnets can reach each other via routes
- └ VPC Endpoints: Private → S3/DynamoDB without internet

COST BREAKDOWN:

- | └ NAT Gateway: \$0.32/hour = \$232/month (if left running 24/7)
- | └ Data transfer via NAT: \$0.02/GB
- | └ VPC Endpoints (S3, DynamoDB): FREE
- | └ Secrets Manager Endpoint: ~\$7/month
- | └ Internet Gateway: FREE
- | └ VPC itself: FREE

└─ Security Groups: FREE
└─ Route Tables: FREE
└─ Total: ~\$240/month (if NAT runs 24/7)

NEXT STEPS:

1. Lab 1.3 will create S3 bucket in this VPC
2. Future labs will launch RDS in private-subnet-1a
3. Future labs will launch EC2/Glue in private-subnet-1b
4. DELETE NAT Gateway when not in use (saves \$232/month!)

SUCCESS CRITERIA - VERIFY YOU'RE DONE

Check off each item:

- ☐ VPC "data-platform-vpc" created with 10.0.0.0/16 CIDR
- ☐ 3 subnets created (1 public, 2 private in different AZs)
- ☐ Internet Gateway created and attached
- ☐ Elastic IP allocated
- ☐ NAT Gateway created and available
- ☐ Public route table created with route to IGW
- ☐ Private route table created with route to NAT
- ☐ Both private subnets associated with private route table
- ☐ Public subnet associated with public route table
- ☐ 3 Security Groups created with correct rules
 - ☐ sg-public-nat with HTTPS rule
 - ☐ sg-private-compute with internal traffic rules
 - ☐ sg-private-db with MySQL/PostgreSQL rules
- ☐ 3 VPC Endpoints created and available
 - ☐ S3 Gateway Endpoint
 - ☐ DynamoDB Gateway Endpoint
 - ☐ Secrets Manager Interface Endpoint
- ☐ All resources verified in AWS console
- ☐ Documentation file saved with all IDs

If all checked ✓, you've completed Lab 1.2!

PART 11: COMPLETE TEARDOWN (Cost Saving)

IMPORTANT: NAT Gateway costs \$0.32/hour. DELETE it when not in use!

Step 11.1: Delete NAT Gateway

1. Click "NAT gateways" in left sidebar
2. Click on your NAT gateway `data-platform-nat`
3. Click "Delete NAT gateway" button (top right)
4. Confirmation message: Type `delete`
5. Click "Delete" (red button)

Expected Result: NAT Gateway status shows "Deleting" then disappears

Cost saved: \$230-240/month if running 24/7!

Step 11.2: Release Elastic IP

1. Click "Elastic IPs" in left sidebar
2. Find your Elastic IP (the one you allocated)
3. Click on it to select it
4. Click "Release Elastic IP address" button
5. Confirm: Click "Release"

Expected Result: Elastic IP is released back to AWS pool

Step 11.3: What NOT to Delete

KEEP these (used in Lab 1.3):

- ✓ VPC (data-platform-vpc)
- ✓ Subnets (all 3)
- ✓ Internet Gateway (needed for VPC)
- ✓ Route Tables (need to exist)
- ✓ Security Groups (will use in future)
- ✓ VPC Endpoints (free!)

Only DELETE:

- ✗ NAT Gateway (costs money)
- ✗ Elastic IP (associated with NAT)

Step 11.4: Cost Verification

1. **Go to:** AWS Billing dashboard
2. **Check:** Current charges for today
3. **You should see:**
 - EC2: ~\$0 (no instances running)
 - VPC: \$0 (VPC itself is free)
 - NAT: \$0 (deleted)
 - **Total: \$0 or minimal**

If costs look high, check:

- NAT Gateway still running? (Delete it!)
 - EC2 instance running? (Terminate it!)
 - Expensive volume types? (Delete them!)
-

WHAT YOU LEARNED

- **VPC Fundamentals**
 - What a VPC is (virtual private cloud)
 - Why companies use VPCs (security, isolation)
 - Public vs. private subnets
- **Network Security**
 - Principle of layered security
 - Why databases should never be public
 - Security groups as firewalls
- **AWS Networking Components**
 - Internet Gateway (connect to internet)
 - NAT Gateway (outbound internet, inbound blocked)
 - VPC Endpoints (cheap alternative to NAT)
 - Route Tables (GPS for traffic)
 - Subnets (network segments)
- **Cost Optimization**
 - NAT Gateway is expensive (\$232/month)
 - VPC Endpoints are cheap (FREE or \$7/month)
 - Understanding trade-offs
- **Redundancy**
 - Why use 2 Availability Zones
 - How to survive AZ failure
 - AWS best practices